

Punto 3.9 — Lazy Loading: configuración, funcionamiento y riesgos

Contexto del proyecto

Este checkpoint continúa `M03/PROYECTO/3.8`. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Lazy Loading: configuración, funcionamiento y riesgos** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.9
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerTodasSinInclude`, `IgnoreAutoIncludes`, `UseLazyLoadingProxies`, `virtual`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/CargaLazyUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class CargaLazyUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public CargaLazyUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== LAZY LOADING (DEMOSTRACIÓN) ===");
        var ordenes = _unidad.Ordenes.ObtenerTodasSinInclude();
        var totalPlanchas = 0;
        foreach (var orden in ordenes)
            totalPlanchas += orden.Planchas.Count;
        if (ordenes.Count != 5 || totalPlanchas != 5) throw new InvalidOperationException("Lazy Loading
no cargó las relaciones.");
        Console.WriteLine($"Órdenes: {ordenes.Count} | Planchas accedidas bajo demanda:
{totalPlanchas}");
        Console.WriteLine("Advertencia docente: el acceso dentro de un bucle puede producir N+1
consultas.");
    }
}

```

Explicación línea a línea del caso de uso 3.9

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial IUnidadDeTrabajo e IOrdenRepositorio, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class CargaLazyUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de DbContext.

Línea 8: `public CargaLazyUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== LAZY LOADING (DEMOSTRACIÓN) ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var ordenes = _unidad.Ordenes.ObtenerTodasSinInclude();` → Obtiene las órdenes neutralizando AutoInclude; así el acceso posterior a Planchas puede demostrar realmente Lazy Loading.

Línea 14: `var totalPlanchas = 0;` → Inicializa el acumulador que permitirá comprobar cuántas planchas se cargan al acceder a las navegaciones Lazy.

Línea 15: `foreach (var orden in ordenes)` → Recorre los resultados para evaluar el comportamiento de cada entidad o relación.

Línea 16: `totalPlanchas += orden.Planchas.Count;` → Accede a la navegación virtual Planchas de cada orden y acumula su tamaño; en 3.9 este acceso es precisamente el que puede disparar una consulta Lazy por entidad.

Línea 17: `if (ordenes.Count != 5 || totalPlanchas != 5) throw new InvalidOperationException("Lazy Loading no cargó las relaciones.");` → Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `Console.WriteLine($"Órdenes: {ordenes.Count} | Planchas accedidas bajo demanda: {totalPlanchas}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 19: `Console.WriteLine("Advertencia docente: el acceso dentro de un bucle puede producir N+1 consultas.");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<CargaLazyUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.9 OK");
```

Explicación línea a línea de Program.cs 3.9

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<CargaLazyUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<CargaLazyUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.9 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.9 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

En 3.9, los proxies y las navegaciones `virtual` existen solo para demostrar Lazy Loading y el riesgo N+1. El siguiente checkpoint vuelve a desactivar la configuración de proxies.

Paso 9: Diagnosticar un error común

Error: Acceder a navegaciones Lazy dentro de un bucle puede producir N+1 consultas y depender de un DbContext aún vivo.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: `APOYO M03 3.9 - PROXIES FRESCOS SIN INCLUDE`

```
// Limpia el tracker y devuelve entidades proxy sin cargar Planchas.
List<OrdenFabricacion> ObtenerTodasSinIncludeLimpiasReto();
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs

Marcador: APOYO M03 3.9 - PROXIES FRESCOS SIN INCLUDE

```
// Se usa sólo en copias pedagógicas para aislar Lazy Loading del estado previo del ChangeTracker.
public List<OrdenFabricacion> ObtenerTodasSinIncludeLimpiasReto()
{
    _context.ChangeTracker.Clear();
    return _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Archivo: src/AceriaData.Application/CargaLazyUseCase.cs

Marcador: ERROR CONTROLADO M03 3.9 - LAZY LOADING CON DBCONTEXT CERRADO

```
// Demuestra el Paso 9: el proxy necesita que el DbContext siga vivo.
var fueraDeContexto = _unidad.Ordenes.ObtenerTodasSinIncludeLimpiasReto();
_unidad.Dispose();

// Esta línea debe fallar: la navegación aún no se ha cargado y el contexto ya está cerrado.
var planchasFueraDeContexto = fueraDeContexto[0].Planchas.Count;
Console.WriteLine($"No debería alcanzarse: {planchasFueraDeContexto}");
```

Comprobación esperada: Lazy Loading depende de un DbContext vivo y falla fuera de su ámbito.

Laboratorio adicional del punto 3.9

Diagnóstico técnico

Lazy Loading necesita Proxies, `UseLazyLoadingProxies()` y navegaciones virtuales, además de un DbContext vivo. Como 3.8 introdujo `AutoInclude`, `ObtenerTodasSinInclude()` usa `IgnoreAutoIncludes()` para que la navegación se cargue realmente bajo demanda.

Reto resuelto y verificación adicional

Recorre las cinco órdenes y accede a `orden.Planchas.Count`; el total esperado es cinco. Contrasta este acceso con Eager Loading y explica por qué el patrón puede producir N+1 al crecer N.

Errores comunes revisados

Error	Causa	Solución
Proxy Lazy no intercepta una navegación	La navegación que se quiere cargar bajo demanda no puede ser sobrescrita por el proxy	En el modelo basado en proxies, declarar <code>virtual</code> las navegaciones destinadas a Lazy Loading
Carga Lazy no funciona	No se configuró <code>UseLazyLoadingProxies</code>	Configurar en <code>OnConfiguring</code>
N+1 en bucle	Con Lazy Loading se accede repetidamente a navegaciones todavía no cargadas	Planificar Eager Loading, proyección o Explicit Loading según el caso
<code>ObjectDisposedException</code>	Se accede a propiedades de navegación fuera del ámbito	Usar Include o DTOs
Referencia circular en serialización	Las entidades se referencian mutuamente	Usar DTOs o configurar el serializador
<code>NullReferenceException</code>	La propiedad de navegación es null	Comprobar antes de acceder

Error	Causa	Solución
Proxies no generados	Las clases son sealed	Quitar sealed

Analogía operativa

La carga Lazy en una acería es como pedirle al archivo central que no te traiga las carpetas relacionadas hasta que las pidas. En lugar de recibir la carpeta de la orden con las planchas y el detalle adjuntos, recibes solo la carpeta de la orden. Si después necesitas las planchas, el archivo central te las trae en ese momento. Si necesitas el detalle, te lo trae después. Cada petición adicional es un viaje al archivo central. Si tienes cien órdenes y pides las planchas de cada una, haces cien viajes. Eso es el problema N+1. La carga Eager es como pedir todas las carpetas relacionadas de una vez: un solo viaje con todo lo necesario. La carga Lazy es cómoda porque no tienes que decidir de antemano qué necesitas, pero es ineficiente si haces muchas peticiones. En una acería con mucho volumen, la carga Eager es la opción correcta. En un prototipo o en una aplicación pequeña, la carga Lazy puede ser aceptable. Así funciona la carga Lazy en EF Core: cómoda pero peligrosa, útil en prototipos pero ineficiente en producción.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Cuenta cuántas navegaciones se acceden en el bucle y razona cuántas consultas puede provocar Lazy Loading.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos de forma coordinada sólo para esta variante y restaura después el estado original.

Archivo: src/AceriaData.Application/Interfaces.cs

Marcador: APOYO M03 3.9 - PROXIES FRESCOS SIN INCLUDE

```
// Limpia el tracker y devuelve entidades proxy sin cargar Planchas.
List<OrdenFabricacion> ObtenerTodasSinIncludeLimpiasReto();
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs

Marcador: APOYO M03 3.9 - PROXIES FRESCOS SIN INCLUDE

```
// Se usa sólo en copias pedagógicas para aislar Lazy Loading del estado previo del ChangeTracker.
public List<OrdenFabricacion> ObtenerTodasSinIncludeLimpiasReto()
{
    _context.ChangeTracker.Clear();
    return _context.OrdenesFabricacion
        .IgnoreAutoIncludes()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}
```

Archivo: src/AceriaData.Application/CargaLazyUseCase.cs

Marcador: RETO M03 3.9 - CONTAR ACCESOS DE NAVEGACION Y RIESGO N+1

```
// Parte de proxies frescos sin AutoInclude y accede una vez a Planchas por cada orden.
var ordenesReto = _unidad Ordenes.ObtenerTodasSinIncludeLimpiasReto();
var accesosNavegacion = 0;
var totalReto = 0;

foreach (var orden in ordenesReto)
{
    accesosNavegacion++;
    totalReto += orden.Planchas.Count;
}

if (ordenesReto.Count != 5 || accesosNavegacion != 5 || totalReto != 5)
    throw new InvalidOperationException("Reto 3.9: accesos Lazy o cardinalidad inesperados.");

Console.WriteLine($"Reto 3.9 OK | Órdenes: {ordenesReto.Count} | Accesos navegación:
{accesosNavegacion} | Planchas: {totalReto} | Consultas potenciales: 1 + {accesosNavegacion}");
```

Comprobación esperada: cinco accesos de navegación hacen visible el riesgo potencial $1 + N$.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar 3.9 OK, el siguiente estado parte exactamente de esta solución y añade **Explicit Loading**.
